



Core Infrastructure (`app/core/`)

Coresight Research Portal — Technical Documentation

Document version: 1.0

Generated: June 29, 2026

Classification: Internal — Engineering Reference

CONFIDENTIAL — INTERNAL USE ONLY — DO NOT DISTRIBUTE

This document contains proprietary information belonging to Coresight Research. Unauthorized disclosure, copying, or distribution is strictly prohibited.

Table of Contents

Core Infrastructure (app/core/)	1
Overview	3
Module Index	4
config.py — Application Configuration	4
database.py — Dual-Engine Database Access	6
auth_manager.py — Session-Token Authentication	8
access_control.py — Page ACL & System Roles	11
auth_environment.py — OIDC Gate	13
ssl_setup.py — Azure MySQL SSL CA	13
boot_overlay.py — Streamlit Boot Loader	14
llm_extractor.py — GPT Filing Metric Extraction	14
search_engine.py — TF-IDF Transcript Search	16
Startup Integration (main.py)	17
Quick Reference: Import Map	18

Application: Market Data Portal (MDP)

Stack: Streamlit + SQLAlchemy 2 + Azure MySQL + OpenID Connect (OIDC)

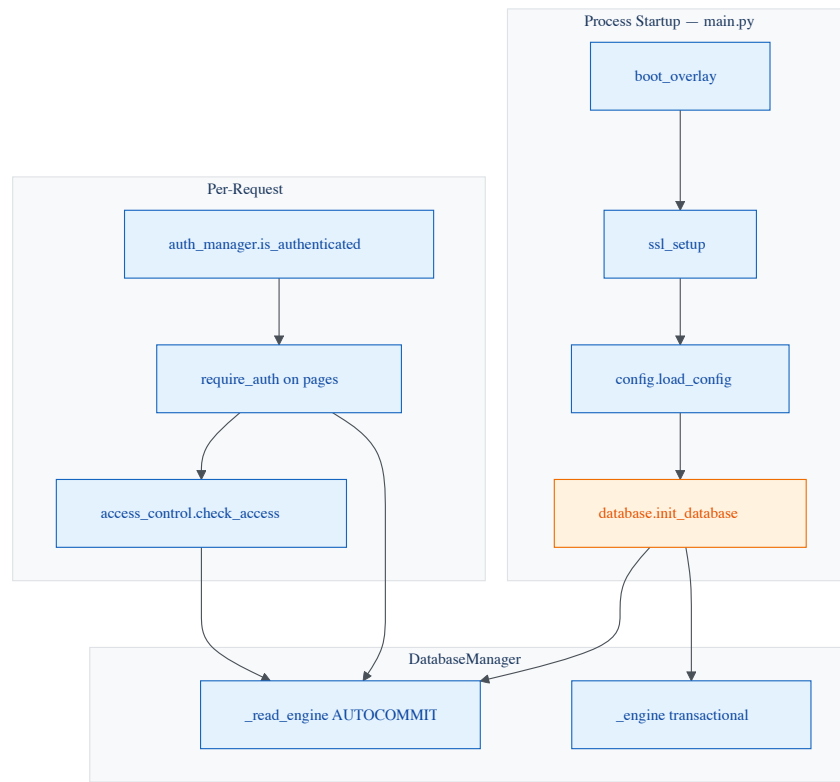
Scope: app/core/ — configuration, database, authentication, access control, Large Language Model (LLM) extraction, search, SSL, boot overlay

Overview

The app/core/ package is the application infrastructure layer. It sits below pages (app/pages/) and the data layer (app/data/), providing:

- Environment-based configuration and SSL
- Dual-engine database access (read vs transactional)
- OIDC session-token authentication (cookie-first)
- Page-level and system-role access control
- On-demand LLM filing metric extraction
- In-memory Term Frequency-Inverse Document Frequency (TF-IDF) transcript search (utility, currently unwired)

- In-memory Term Frequency-Inverse Document Frequency (TF-IDF) transcript search (utility, currently unwired)



Module Index

Module	Purpose	Primary consumers
<code>config.py</code>	Env-based <code>AppConfig</code> / <code>DatabaseConfig</code>	<code>database.py</code> (indirect: entire app)
<code>database.py</code>	Singleton <code>DatabaseManager</code> , dual engines	30+ modules across pages/data/utils
<code>auth_manager.py</code>	Cookie-first OIDC session auth	All protected pages, <code>navigation</code> , <code>main.py</code>
<code>access_control.py</code>	Page Access Control List (ACL) + system roles	ACL pages, <code>navigation</code> , <code>forecast_refresh_service</code>
<code>auth_environment.py</code>	OIDC/production flags	<code>auth_manager</code> , <code>main</code> , <code>login</code> , <code>logout_bridge</code>
<code>ssl_setup.py</code>	DigiCert CA provisioning	<code>main.py</code> , <code>config.py</code>
<code>boot_overlay.py</code>	Streamlit index.html patch	<code>main.py</code>
<code>llm_extractor.py</code>	GPT-4o-mini filing extraction	<code>repository.py</code> , <code>company_filings.py</code>
<code>search_engine.py</code>	TF-IDF segment search	Unused (orphan utility)
<code>__init__.py</code>	Package marker	—

`config.py` — Application Configuration

Classes

Class	Role
<code>Environment</code> (Enum)	<code>LOCAL</code> , <code>STAGING</code> , <code>PRODUCTION</code>
<code>DatabaseConfig</code> (dataclass)	MySQL connection params, pool settings, SSL
<code>AppConfig</code> (dataclass)	<code>env</code> , <code>debug</code> , <code>database</code> , <code>secret_key</code> , session timeout, caching, pagination

Key functions

Function	Role
<code>load_config()</code>	Build <code>AppConfig</code> from environment
<code>DatabaseConfig.connection_string</code>	<code>mysql+pymysql://...</code> property
<code>DatabaseConfig.connect_args</code>	SSL <code>{"ssl": {"ca": path}}</code> with late-binding CA resolution

Environment variables

Variable	Default / notes
<code>APP_ENV</code>	<code>local</code> → drives <code>Environment</code> enum
<code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_NAME</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code>	Local fallbacks
<code>STG_DB_*</code>	Staging Azure MySQL
<code>PROD_DB_*</code>	Production Azure MySQL
<code>ENABLE_SSL</code>	<code>true</code> (stg/prod), <code>false</code> (local)
<code>SSL_CA</code>	Optional; late-bound via <code>ssl_setup</code> if missing
<code>DB_POOL_SIZE</code>	5
<code>DB_MAX_OVERFLOW</code>	10
<code>DEBUG</code>	<code>true</code> local; <code>false</code> prod
<code>SECRET_KEY</code>	Warns if default in non-local
<code>SESSION_TIMEOUT</code>	3600
<code>ENABLE_CACHING</code>	<code>true</code>
<code>CACHE_TTL</code>	300
<code>NAVIGATION_MODE</code>	same

Global singleton: `config = load_config()` at module import.

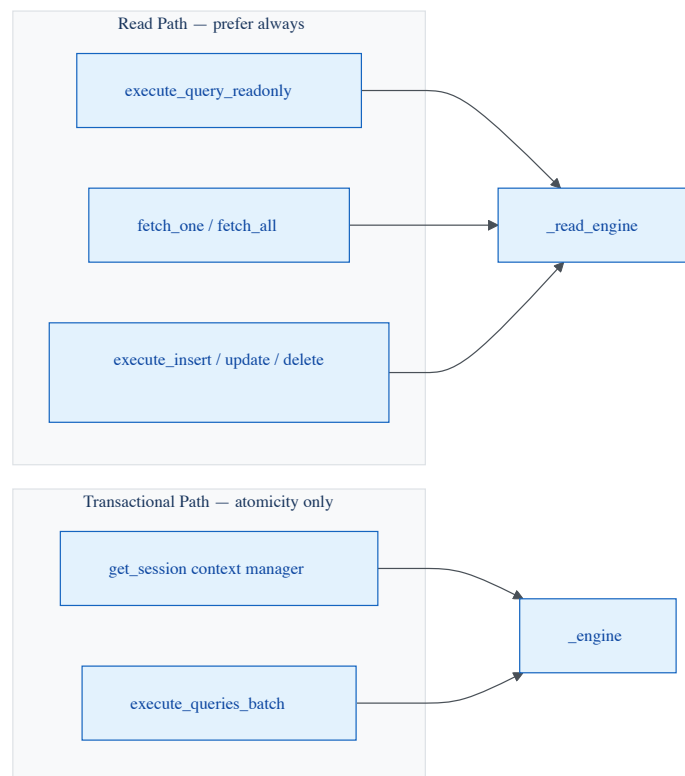
database.py — Dual-Engine Database Access

Design philosophy

On Azure MySQL (~250ms round-trip per connection operation), minimizing RTTs is critical:

Engine	Attribute	Isolation	Use when	Azure RTTs
Read	<code>_read_engine</code>	<code>AUTOCOMMIT</code> + <code>skip_autocommit_rollback=True</code>	All SELECTs, single-statement writes	~1 RTT
Main	<code>_engine</code>	Default (implicit BEGIN)	Multi-statement atomicity, <code>get_session()</code>	~3 RTTs

On Azure MySQL (~250ms round-trip per connection operation), minimizing RTTs is critical:



DatabaseManager Application Programming Interface (API)

Method	Engine	Returns	Notes
<code>connect()</code>	Both	—	Creates engines; parallel SSL warmup (6 conns on Azure)
<code>get_session()</code>	Main	Context manager	commit/rollback
<code>execute_query_readonly(query, params)</code>	Read	<code>List[Dict]</code>	Swallows errors → <code>[]</code>
<code>execute_query_readonly_raising(...)</code>	Read	<code>List[Dict]</code>	Raises on error (cache-safe)
<code>fetch_one</code> / <code>fetch_all</code>	Read	<code>Optional[Dict]</code> / <code>List[Dict]</code>	Thin wrappers
<code>execute_insert</code> / <code>execute_update</code> / <code>execute_delete</code>	Read (AUTOCOMMIT)	<code>int</code> rowcount	Single-statement writes
<code>execute_insert_returning_id</code>	Read	<code>Optional[int]</code>	<code>lastrowid</code>
<code>execute_scalar</code>	Main session	scalar	
<code>execute_queries_batch</code>	Main session	—	Multi-query, one connection
<code>health_check()</code>	Main	<code>bool</code>	<code>SELECT 1</code>
<code>_dispose_and_reconnect()</code>	Both	—	SSL recovery

Pool tuning (Azure)

- `pool_pre_ping=False` on Azure — avoids 250ms `SELECT 1` per checkout
- `pool_recycle=3600` — avoids 4s SSL re-handshake on stale connections

Module-level exports

Symbol	Role
<code>db_manager</code>	Global <code>DatabaseManager()</code> singleton
<code>with_db_session</code>	Decorator injecting <code>session</code> kwarg
<code>init_database()</code>	<code>db_manager.connect()</code> — fail fast
<code>warmup_av_fulltext()</code>	Gated by <code>ENABLE_AV_FT_WARMUP=1</code> ; warms FULLTEXT indexes

Usage pattern (from project conventions)

```
# READ (1 RTT — always prefer):
rows = db_manager.execute_query_readonly(query, params)
row = db_manager.fetch_one(query, params)

# WRITE (1 RTT — AUTOCOMMIT):
db_manager.execute_insert(query, params)
db_manager.execute_update(query, params)

# TRANSACTIONAL (3 RTTs — only for atomicity):
with db_manager.get_session() as session:
    session.execute(text(query), params)
```

`auth_manager.py` — Session-Token Authentication

Design principle

Cookie is the source of truth. `is_authenticated()` never hits the database.
`market_data_user_sessions` is audit-only on login.

Constants

Name	Value
<code>COOKIE_NAME</code>	<code>"auth_session"</code>
<code>COOKIE_MAX_AGE_DAYS</code>	<code>7</code>
<code>LOGOUT_BRIDGE_COOKIE_NAME</code>	<code>"auth_logout_bridge"</code>

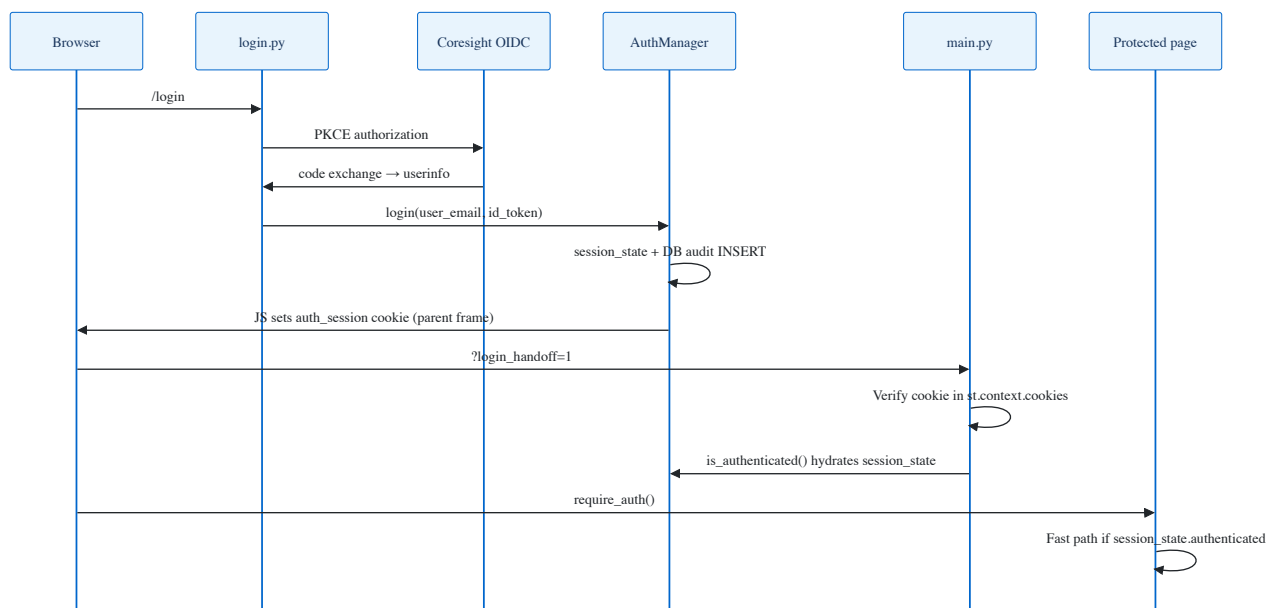
Classes

Class	Role
<code>AuthResult</code>	<code>success</code> , <code>session_id</code> , <code>user_email</code> , <code>error_message</code>
<code>AuthManager</code>	Login, auth check, logout, cookie I/O

Auth flow

End-to-end OIDC login: Identity Provider (IdP) authorization, cookie handoff via parent-frame JavaScript, and per-page identity checks.

End-to-end OIDC login: Identity Provider (IdP) authorization, cookie handoff via parent-frame JavaScript, and per-page identity checks.



require_auth() check order

1. `st.session_state._auth_invalidated` → redirect login
2. `st.session_state.authenticated` + `auth_data` → fast path
3. `_get_cookie()` from `st.context.cookies` → `CookieController` fallback
4. Blacklist check (`_is_session_invalidated`)
5. Retry with `AUTH_RETRY_DELAYS` (default `0.15, 0.35` s) → `st.rerun()`
6. Exhausted → `st.switch_page(login)`

Cookie design

Slim cookie JSON only: `session_id`, `user_email`, `user_display_name`, `login_at`

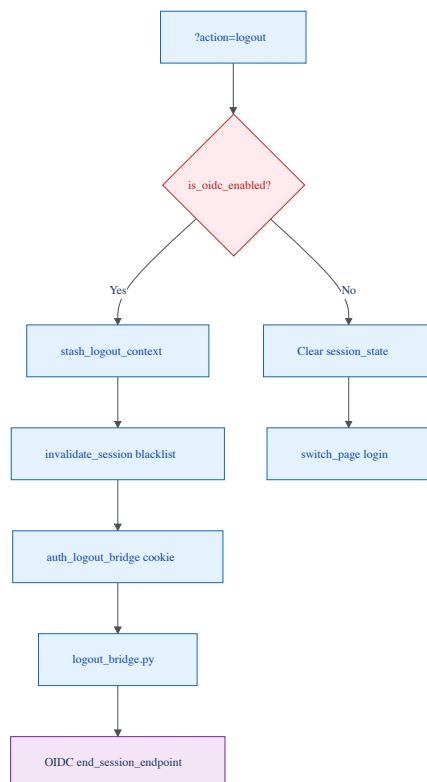
Not in cookie: `id_token` stored in `/tmp/_oidc_idtok_{hash20}.json` and `session_state.auth_data`

Cookie write: Parent-frame JS injection (proven on Azure STG; `CookieController.set` unreliable cross-frame)

Logout flow

Logout routing branches on OIDC mode — stash context and RP-initiated IdP logout, or legacy session clear.

Logout routing branches on OIDC mode — stash context and RP-initiated IdP logout, or legacy session clear.



Module-level exports

```
get_auth_manager , login_user , stash_logout_context , require_auth , is_authenticated ,
logout , get_current_user , get_session_id , get_auth_data , get_current_domain ,
build_cookie_clear_js_lines , render_auth_cookie_handoff_redirect
```

Environment variables

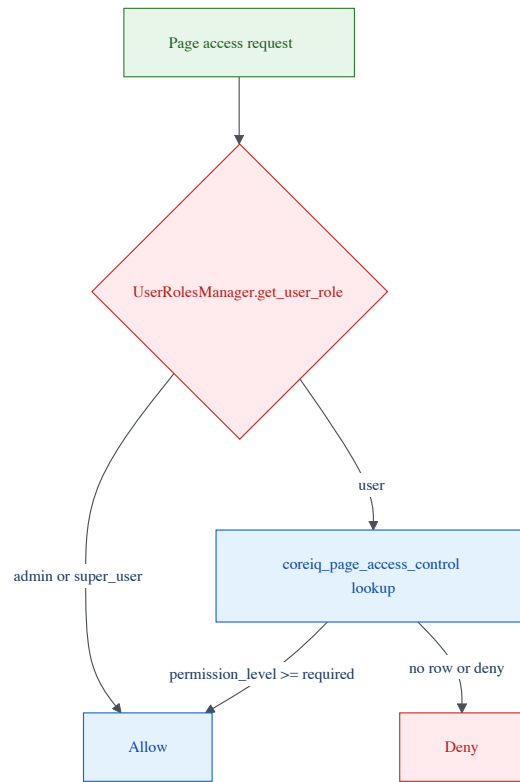
Variable	Role
APP_ENV	Deployment environment (LOCAL , STAGING , PRODUCTION)
DEBUG	Verbose diagnostics
AUTH_FLOW_VERBOSE	Verbose auth timing logs
AUTH_RETRY_DELAYS	Cookie retry schedule
AUTH_COOKIE_SYNC_WAIT_MS / AUTH_COOKIE_SYNC_POLL_MS	Handoff timing
AUTH_LOGOUT_BRIDGE_*	Bridge cookie sync

access_control.py — Page ACL & System Roles

Two-layer model

Access checks consult system role first (admin bypass), then page-level ACL rows in MySQL.

Access checks consult system role first (admin bypass), then page-level ACL rows in MySQL.



AccessControlManager

Method	Role
<code>check_access(page_name, user_email, required_permission)</code>	Hierarchy check; admins bypass
<code>get_page_users(page_name)</code>	List ACL rows
<code>set_user_access(...)</code>	UPSERT permission
<code>remove_user_access(...)</code>	DELETE ACL row

Permission hierarchy: `view_only(1) < upload_only(2) < edit(3) < delete_admin(4) = allow(4)`.
 Explicit `deny` blocks all.

Table: `coreiq_page_access_control`

UserRolesManager

Method	Role
get_user_role / set_user_role / delete_user	Create, Read, Update, Delete (CRUD) on coreiq_user_roles
is_admin / is_admin_or_super_user / can_delete_users	Convenience checks

Roles: admin > super_user > user

Fallback admins (if Database (DB) down): mohdsaeedafri@coresight.com , philipmoore@coresight.com , shashankgupta@coresight.com

Consumers

- Pages: access_management , forecasting , forecasting_admin , admin_iam , audit , reports
- components/navigation.py — footer Access Management link
- data/forecast_refresh_service.py

auth_environment.py — OIDC Gate

Function	Current behavior
is_production_deploy()	True if ENV / ENVIRONMENT / APP_ENV == production
is_oidc_enabled()	Always True (JSON Web Token (JWT) path disabled)
IS_OIDC_ENV	Snapshot at import

ssl_setup.py — Azure MySQL SSL CA

Function	Role
ensure_ca_cert()	Search bundled DigiCert G2 → Azure \$HOME/site/wwwroot/ → CWD → download

Sets `os.environ["SSL_CA"]` on success. Called at startup (`main.py`) and lazily from `DatabaseConfig.connect_args` .

`boot_overlay.py` — Streamlit Boot Loader

Function	Role
<code>patch_streamlit_index_html()</code>	Idempotent patch of Streamlit static <code>index.html</code>

Injects:

- CSS to hide `[data-testid='stSkeleton']`
- Branded Coresight overlay HyperText Markup Language (HTML) + JS
- `MutationObserver` removes overlay when `stMain` contains real widgets
- 15s safety timeout

Marker: `cs-boot-overlay-v1` — re-applied on cold start after Streamlit upgrades.

`llm_extractor.py` — GPT Filing Metric Extraction

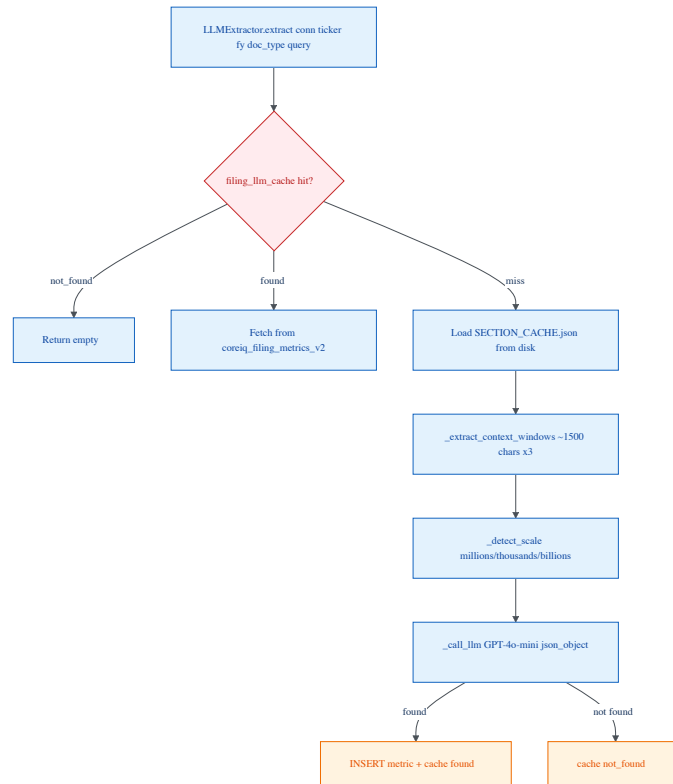
Purpose

On-demand GPT-4o-mini extraction for U.S. Securities and Exchange Commission (SEC) filing metrics missing from eXtensible Business Reporting Language (XBRL). Results cached in `filing_llm_cache` and persisted to `coreiq_filing_metrics_v2` with `source='llm'` .

Extraction flow

LLM metric extraction checks caches first, then builds context windows and calls GPT-4o-mini before persisting results.

LLM metric extraction checks caches first, then builds context windows and calls GPT-4o-mini before persisting results.



Class

Class	Method
LLMExtractor	<code>extract(conn, ticker, fy, doc_type, query) -> List[FilingMetricResult]</code>

Requires: Open SQLAlchemy `conn` from caller (not `db_manager` directly).

Env: `OPENAI_API_KEY` (missing → empty results)

Cost: ~\$0.00012 per unique query; \$0 when cached.

Consumers

- `data/repository.py` — `FilingMetricRepository.search_with_llm_fallback`
- `pages/company_filings.py` — on-demand extraction User Interface (UI)

search_engine.py — TF-IDF Transcript Search

Purpose

In-memory TF-IDF + cosine similarity over earnings call transcript segments (scikit-learn).

Classes

Class	Role
SearchResult	index , speaker , text , score , snippet
TranscriptSearchEngine	__init__(segments) , search(query, top_k, min_score)

Algorithm

1. Fit `TfidfVectorizer` (unigrams+bigrams, 10k features, English stopwords)
2. Cosine similarity ranking
3. Guard: result must contain at least one query word
4. Fallback: keyword count if sklearn missing

Status

Not wired to any page. `earnings_calls.py` uses MySQL `FULLTEXT` via `EarningsCallRepository.search_transcripts_fulltext()` instead.

Quick Reference: Import Map

Module	Imported by
config	database.py only
database	All data services, most pages, auth_manager , access_control , utils
auth_manager	All protected pages, navigation , main , login , logout_bridge
access_control	ACL pages, navigation , forecast_refresh_service
llm_extractor	repository , company_filings
search_engine	— (unused)
auth_environment	auth_manager , main , login , logout_bridge
ssl_setup	main , config
boot_overlay	main , scripts/patch_streamlit_boot.py